From MNIST to distributed training

A self study guide to the project and its path to PBS

A distributed training program is still a training loop. The new work is deciding which process owns which device and data, keeping model updates consistent, and coordinating the parts that should happen only once. This guide follows those decisions through architectures.py, MNIST_demo.py, and utils.py.

Start with one image, trace it through the CNN, and then follow one batch through an optimizer step. Only after that do we add command-line configuration, independent experiments, and cooperating DDP processes. By the end, you should be able to explain both what the current script does and what must be checked before it runs on a PBS cluster.

```
ONE DEVICE              MANY EXPERIMENTS
CNN + training loop  ->  CLI settings + separate jobs
         |                         |
         v                         v
ONE DISTRIBUTED RUN     EVENTUAL HPC EXECUTION
DDP replicas + shards -> PBS allocation + torchrun
```

How to use this guide

Read with the three source files open. Snippets labeled Current code are excerpts from those files; comments in surrounding prose explain them. Suggested additions and conceptual examples are explicitly labeled. They are not changes already present in the project.

The single-device baseline is explained by following the current script's non-distributed branch. The sequence is a learning progression, not a claim that an earlier source version contains a particular implementation.

Reading route

1. Pages 2 to 5 follow the CNN, tensors, data, loss, and training loop.
2. Pages 6 to 7 turn settings into reproducible experiments.
3. Pages 8 to 13 explain process identity, data partitioning, and synchronization.
4. Pages 14 to 15 connect torchrun to eventual PBS execution.
5. Pages 16 to 18 collect fixes, readiness checks, and references.

Current status. The code contains a DDP training path for CUDA and NCCL. Explicit CUDA binding and coordinated data preparation still need attention. Multi-node PBS launching remains dependent on the actual cluster. A code walkthrough does not establish that a distributed run has succeeded on that hardware.

1 The three files and one image

architectures.py defines CNN2D, an nn.Module describing how images become class scores. MNIST_demo.py handles arguments, distributed setup, device selection, model creation, datasets, loaders, training, evaluation, and plotting, in that order. utils.py contains get_distributed_info(), the small bridge between launcher environment variables and Python values.

The separation matters: a new model can replace CNN2D without putting scheduler details inside its forward method. The main script owns the experiment; the helper interprets process identity; the architecture owns tensor computation.

The CNN baseline

MNIST is a handwritten-digit classification task. An input image has one grayscale channel and spatial dimensions 28 by 28. The target is an integer from 0 to 9. After transforms.ToTensor(), a sample image has shape [1, 28, 28], with ordinary MNIST pixel values scaled into [0, 1]. The project applies no additional normalization or augmentation.

Current code from architectures.py

```
self.first_CNN_layer = nn.Conv2d(
    in_channels=1, out_channels=16,
    kernel_size=3, padding=1
)
self.second_CNN_layer = nn.Conv2d(
    in_channels=16, out_channels=32,
    kernel_size=3, padding=1
)
self.dense_1 = nn.LazyLinear(128)
self.dense_2 = nn.Linear(128,10)
```

Each convolution learns small spatial filters. With the default stride of 1 and padding of 1, these 3 by 3 filters preserve height and width. The first produces 16 feature channels; the second produces 32. ReLU follows each convolution and the first dense layer, introducing nonlinearity by replacing negative values with zero.

There is no pooling in this architecture. That detail explains why flattening creates a relatively large vector and why dense_1 contains most of the model's parameters. This is a transparent teaching model, not an assertion that this architecture is the most efficient MNIST classifier.

Pause and predict. Does the second convolution produce 32 classes? No. It produces 32 feature channels. Only the final dense layer maps the learned representation into the ten class scores.

2 Trace the tensor shapes

Let B mean the number of images in the current batch. The DataLoader adds the batch dimension; CNN2D preserves it throughout forward(). A last batch may be smaller than the requested batch size.

Stage in the actual model	Shape	Meaning
Input images	[B, 1, 28, 28]	Batch, channel, height, width
first_CNN_layer and ReLU	[B, 16, 28, 28]	Sixteen feature maps per image
second_CNN_layer and ReLU	[B, 32, 28, 28]	Thirty two feature maps per image
torch.flatten(x, 1)	[B, 25088]	32 times 28 times 28 features
dense_1 and ReLU	[B, 128]	Learned representation
dense_2	[B, 10]	Ten logits for each image
Labels from the loader	[B]	Integer target class per image

Current code from forward()

```
x = torch.flatten(x,1)
x = self.dense_1(x)
x = self.ReLU(x)
x = self.dense_2(x)
return x
```

Starting flatten at dimension 1 keeps separate examples separate. Flattening from dimension 0 would mix the batch with the features and break the intended contract.

Why the dummy forward exists

Current code from MNIST_demo.py

```
model = CNN2D().to(device)
dummy_data = torch.rand(1, 1, 28, 28).to(device)
dummy_output = model(dummy_data)
```

nn.LazyLinear(128) knows its output width but infers its input width on the first forward pass. This dummy input materializes that layer with 25,088 input features. It also checks that a [1, 1, 28, 28] input produces [1, 10]. The script does this before wrapping in DDP, which needs initialized parameters, and before creating Adam.

This initialization does not train the model: no loss, backward call, or optimizer step is performed. A suggested cleanup is to use torch.no_grad() for the dummy pass so it does not retain an unnecessary autograd graph. Changing the input image size later will not make an already initialized dense layer adapt automatically.

Try it mentally. With B = 64, logits have shape [64, 10] and labels have shape [64]. There are 64 classification decisions, each choosing among ten digits.

3 Data loaders logits and loss

The dataset answers “what is sample i?” The DataLoader answers “which samples form the next batch?” MNIST_demo.py creates training and test datasets under ./data with train=True and train=False, respectively. Standard MNIST contains 60,000 training and 10,000 test examples. The training loader shuffles in the single-device branch; the test loader preserves order.

Current code for the training loader

```
train_loader = DataLoader(
    mnist_trainset,
    batch_size=batch_size,
    shuffle=(train_sampler is None),
    sampler=train_sampler
)
```

With no distributed sampler, train_sampler is None and shuffle is True. With a sampler, that sampler owns ordering and partitioning. Specifying two independent shuffling mechanisms would be the wrong abstraction. The current loaders use the default num_workers=0, so loading happens in the training process. Adding worker processes is a later performance choice, not a requirement for correctness. [1]

The diagnostic next(iter(train_loader)) prints batch shapes. It does not permanently remove that batch from training: the epoch loop creates a fresh iterator.

Scores first then a loss

The final layer returns logits: arbitrary real-valued scores, not probabilities. For each image, a larger score means the model favors that class. CrossEntropyLoss accepts these logits directly and, for this task, expects integer class indices in labels. It combines the log-softmax calculation with the classification loss. Do not add a softmax layer before this criterion. [2]

Current code

```
criterion = nn.CrossEntropyLoss()
logits = model(images)
loss = criterion(logits, labels)
```

For [B, 10] logits and [B] integer targets, the default reduction returns one scalar mean loss over the batch. A confidently wrong classification incurs a larger penalty than a correct, confident one. During evaluation, torch.argmax(logits, dim=1) produces [B] predicted class indices. Softmax is unnecessary for argmax because it preserves score order.

Check your understanding. Why not provide a ten-element one-hot target for each image? This code uses the class-index interface: one integer label per image is sufficient. Keep that target contract when adapting the model.

4 One device and one optimizer step

The non-distributed branch prefers CUDA when available, then Apple's MPS backend, then CPU. These are alternative places to execute the same model. This fallback does not make the distributed branch portable to all three backends: that branch explicitly assumes CUDA and NCCL.

Current code for the single-device choice

```
if torch.cuda.is_available():
    device = torch.device("cuda")
elif torch.backends.mps.is_available():
    device = torch.device("mps")
else:
    device = torch.device("cpu")
```

The model is moved with .to(device). Each batch's images and labels must follow it. A device object describes a destination; it does not move existing tensors by itself.

The loop that survives the transition to DDP

Current code inside the batch loop

```
images = images.to(device)
labels = labels.to(device)
logits = model(images)
loss = criterion(logits, labels)
optimizer.zero_grad()
loss.backward()
optimizer.step()
epoch_loss += loss.item()
```

Forward computes scores. The loss measures how those scores disagree with the targets. zero_grad clears gradients left by the previous batch; backward computes derivatives of the loss with respect to trainable parameters; Adam uses those gradients and its own running state to update the parameters. The order makes every batch one fresh optimization step.

model.train() is called before the epoch loop. It selects training behavior for layers that distinguish training from evaluation. The current CNN has no dropout or batch normalization, but keeping this call establishes the correct pattern for future architectures.

loss.item() turns a scalar tensor into a Python number for reporting. It is separate from optimization and may synchronize device work. The reported average is an unweighted average of batch means, so a short final batch has the same weight as a full batch. Section 11 explains a sample-weighted alternative.

Learning experiment. Run a short single-device job first. Confirm image, label, and logit shapes; look for a finite loss and useful accuracy. A successful single-device run gives you a baseline for diagnosing distribution problems, but floating-point results need not be identical across devices.

5 Command line settings make experiments portable

Hard-coded settings force you to edit Python for every experiment. argparse turns the same file into a configurable program: the launcher supplies settings, while the training logic stays in one place. This is particularly useful when a batch scheduler runs a program unattended.

Argument in the current parser	Type	Default
--learning-rate	float	0.001
--epochs	int	10
--batch-size	int	64
--local-rank or --local_rank	int	0

Current code excerpt

```
parser.add_argument(
    "--learning-rate", type=float, default=0.001
)
args = parser.parse_args()
learning_rate = args.learning_rate
```

argparse converts the command-line spelling --learning-rate into args.learning_rate and checks that the value can be parsed as a float. It does not, by itself, reject a negative learning rate or zero batch size; explicit range validation would make failures clearer.

Single-device invocation using existing arguments

```
python MNIST_demo.py --learning-rate 0.001 \
    --epochs 2 --batch-size 64
```

This command assumes the three project files are importable from the working directory and that the required Python packages are installed. The current ./data and incorrect.png paths are relative to that working directory, which becomes important in batch jobs.

Why this matters for PBS and job arrays

A job array can select one configuration per array task and pass those values as arguments. The code no longer needs a separate edited copy for each learning rate. Logs can record the command, configuration, software versions, and output directory so results can be interpreted later.

The parser accepts both local-rank spellings for launcher compatibility, but args.local_rank is not used by the current helper. The environment determines local_rank. Passing --local-rank alone does not turn an ordinary Python invocation into a DDP job.

Suggested follow-up. Add a run identifier, output directory, data directory, and random seed as explicit arguments. They are useful configuration fields, but they are not implemented in the uploaded parser. Do not pass them to the current script until they are added.

6 Sweeps and two kinds of parallel work

A Cartesian product pairs every choice on one axis with every choice on the others. Two learning rates, two batch sizes, and two epoch counts produce 2 times 2 times 2 = 8 configurations. Each configuration should start a fresh model and optimizer. The current script runs one configuration per invocation; it does not contain a sweep loop.

Suggested separate sweep planning code

```
from itertools import product

configs = list(product(
    [0.001, 0.0003],   # learning rates
    [64, 128],         # per-rank batch sizes
    [5, 10],           # epochs
))
for index, (lr, bs, epochs) in enumerate(configs):
    run_name = f"run{index:03d}_lr{lr:g}_bs{bs}_ep{epochs}"
    print(run_name, lr, bs, epochs)
```

This prints a configuration list; it does not submit jobs. A later launcher can select a tuple by an explicitly defined task-index mapping. PBS array indexing and its environment variable must be checked on the cluster. Save the mapping so a job number can be traced back to its settings.

Independent jobs versus one cooperating job

```
JOB LEVEL PARALLELISM             DDP WITHIN ONE RUN
Run A: lr 0.001 -> model A        rank 0 -> data shard 0
Run B: lr 0.0003 -> model B       rank 1 -> data shard 1
No shared gradients              shared gradient updates
Different experiments            replicas of one model
```

Job-level parallelism explores different settings. DDP divides one experiment's training work among processes. You can combine them: each sweep configuration can be a separate DDP job. All ranks within that job must use compatible model structure, settings, and control flow.

Make outputs belong to a run

A useful directory name includes a unique run ID, learning rate, per-rank batch size, epoch count, world size, and seed. World size changes the effective global batch; a seed distinguishes repetitions. An index alone is meaningful only alongside its saved configuration manifest.

The current plotting code always writes incorrect.png. Concurrent or repeated runs in one directory can overwrite it. A run directory and configurable save path are proposed additions. If you instead change the working directory per job, also resolve the script and shared data paths deliberately; otherwise ./data changes too.

7 Learn the process identities

DDP usually runs one Python process per GPU for this design. Each process executes the same MNIST_demo.py, creates its own model and optimizer, and receives its own data batches. The three identity values answer different questions.

Name	Meaning	Example with two nodes and two GPUs each
rank	Process number across the whole job	0, 1, 2, 3
local_rank	Process number within its node	0, 1 on each node
world_size	Total participating processes	4 everywhere

Use global rank for “only once in the entire run.” Use local rank for the GPU index on that node. Checking local_rank == 0 would select one process on every node, not one process for the whole job.

Current helper in utils.py

```
def get_distributed_info():
    distributed = "RANK" in os.environ and "WORLD_SIZE" in os.environ
    if distributed:
        rank = int(os.environ["RANK"])
        local_rank = int(os.environ["LOCAL_RANK"])
        world_size = int(os.environ["WORLD_SIZE"])
    else:
        rank = 0
        local_rank = 0
        world_size = 1
    return distributed, rank, local_rank, world_size
```

The environment is the launcher's message to the script. An ordinary Python run normally lacks these variables, so the helper returns a single-process default. torchrun supplies them for its workers. [3]

Subtle behavior. The helper tests whether RANK and WORLD_SIZE exist, not whether world_size is greater than one. A one-worker torchrun invocation still enters the distributed path, which uses CUDA and NCCL. It also assumes LOCAL_RANK exists if the other two do; a partially configured environment can raise KeyError. Malformed integer values can fail conversion.

Follow the numbers

```
Node A                    Node B
GPU 0: rank 0, local 0     GPU 0: rank 2, local 0
GPU 1: rank 1, local 1     GPU 1: rank 3, local 1
          WORLD_SIZE = 4 for every worker
```

This is a homogeneous two-GPU-per-node illustration. Do not infer a physical GPU number from global rank: rank 2 here uses the first visible GPU on Node B.

8 Bind the GPU and connect the workers

A process group defines the participating processes and the communication backend they use. NCCL supplies GPU communication for the CUDA path here. init_method="env://" reads the rendezvous configuration from the environment; workers need consistent rank/world information and a reachable coordinator. Initializing the group does not create the worker processes. [4]

Current code near the start of MNIST_demo.py

```
if distributed:
    dist.init_process_group(
        backend="nccl",
        init_method="env://"
    )
```

Later, the script chooses torch.device(f"cuda:{local_rank}"). That puts explicitly moved tensors on the intended GPU, but the script never sets the process's current CUDA device. Add torch.cuda.set_device(local_rank), ideally before initializing NCCL or using CUDA. This makes the process-to-device ownership explicit, including operations that rely on the current device. [4]

Suggested replacement for distributed startup

```
distributed, rank, local_rank, world_size = get_distributed_info()
if distributed:
    if not torch.cuda.is_available():
        raise RuntimeError("This DDP path requires CUDA")
    torch.cuda.set_device(local_rank)
    device = torch.device("cuda", local_rank)
    dist.init_process_group(backend="nccl", init_method="env://")
else:
    # Retain the existing CUDA / MPS / CPU selection here.
    ...
```

This is an annotated integration pattern, not a complete replacement script. Move the existing device branch into this location and remove the earlier duplicate initialization rather than initializing the process group twice.

Visible GPU numbering

local_rank selects within the GPUs visible to that process. If a scheduler masks GPUs, cuda:0 can refer to a device with a different physical machine index. Verify visibility and the cluster's launch convention before assuming that local rank maps directly to a hardware label. Each worker in this design needs exclusive access to its assigned GPU.

Failure clue. If workers unexpectedly use the same GPU, inspect LOCAL_RANK, CUDA_VISIBLE_DEVICES, and torch.cuda.current_device() from each worker. If initialization stalls, inspect worker counts and rendezvous connectivity as well; tensor placement alone cannot repair a missing worker.

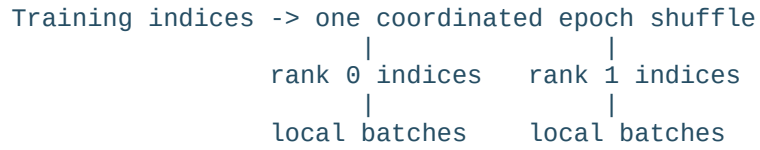
9 Give each rank its share of data

DDP coordinates gradients, but does not split the DataLoader's inputs for you. The current DistributedSampler gives each rank a subset of training indices. All ranks construct the dataset; the sampler decides which indices each rank iterates. This differs from loading the whole dataset into GPU memory. [5]

Current code

```
train_sampler = DistributedSampler(
    mnist_trainset,
    num_replicas=world_size,
    rank=rank,
    shuffle=True
)
```

The loader's shuffle flag becomes False when this sampler exists. At the start of every epoch, the script calls train_sampler.set_epoch(epoch). That changes the shared shuffle order across epochs while keeping partitions coordinated. Omitting it repeats the sampler's ordering across epochs. [1]



With the default sampler drop_last=False, padding can repeat a few indices to give every rank the same number of samples when the dataset size is not divisible by world_size. Do not promise strictly disjoint data in every case. The DataLoader also defaults to drop_last=False, so it retains a final partial local batch. These are two different places where “drop last” can matter. [1]

Per rank is not global

--batch-size is passed to every rank's DataLoader. With equal full local batches and no gradient accumulation, the global batch per optimizer step is batch_size times world_size. A batch size of 64 with four workers therefore uses 256 examples per full step.

For the current 60,000-example training set, four ranks each receive 15,000 examples. With local batch size 64, each rank takes 235 steps per epoch: 234 full batches and a final batch of 24. The last global batch is 96, not 256. A single rank with batch size 64 takes 938 steps.

Experiment design consequence. Changing GPU count while holding per-rank batch size fixed changes batch size and steps per epoch. Record both. Holding the global batch fixed requires adjusting local batch size where divisible; learning-rate changes should be evaluated, not assumed to be universally necessary.

10 DDP changes what backward does

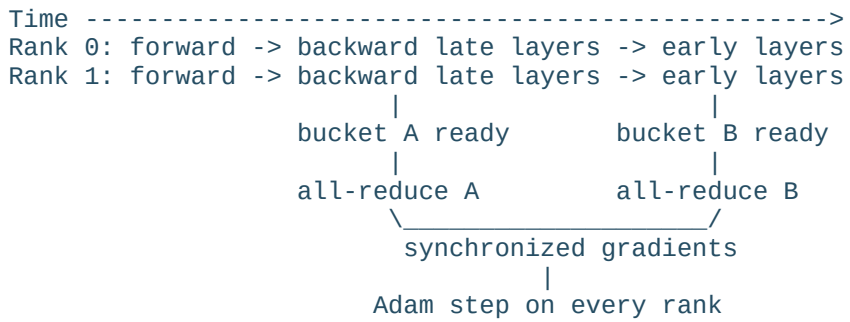
After the dummy forward initializes the lazy layer, the script wraps the model and then creates Adam from model.parameters(). Each worker has a full replica; DDP does not divide the model layers across devices.

Current code

```
if distributed:
    model = DDP(model, device_ids=[local_rank])
optimizer = torch.optim.Adam(
    model.parameters(), lr=learning_rate
)
```

DDP initializes replicas consistently and installs autograd hooks that arrange gradient communication. The ordinary-looking model(images), loss.backward(), optimizer.step() sequence now participates in a coordinated update. With the default behavior, gradients are averaged across workers; the optimizer still runs separately on each worker. DDP does not compute a global scalar loss for your logging. [5]

A timeline for one batch



“All-reduce” means combining corresponding values across ranks and making the result available to all of them. For DDP's standard averaging, a toy gradient of 2 on one rank and 6 on another becomes 4 on both. With equal local batch sizes and mean loss, this corresponds to a mean over the combined batch, subject to ordinary numerical differences.

Gradients become available as autograd walks backward through the model. DDP groups them into buckets. A ready bucket can begin communication while computation continues for earlier layers, overlapping communication with backward work. It does not need to wait for every parameter's gradient before starting all communication. The amount of useful overlap depends on the model, bucket layout, hardware, and network. [6]

Why replicas stay together. Starting with the same parameters, receiving the same averaged gradients, and applying matching optimizer updates keeps replicas aligned. Different settings or incompatible collective call order can break that reasoning. This small CNN may be too small to hide communication cost; more GPUs do not guarantee a faster MNIST demonstration.

11 A reported loss is a separate statistic

Current code after each local training epoch

```
average_loss = epoch_loss / len(train_loader)
if rank == 0:
    print(
        f"Epoch {epoch + 1}/{num_epochs} "
        f"- Loss: {average_loss:.4f}"
    )
```

Every rank has its own epoch_loss. The rank-0 condition suppresses duplicate epoch messages, but does not aggregate those values. The printed number is rank 0's average of local batch losses. DDP gradient synchronization does not turn it into a globally reduced epoch loss.

There is a second distinction: averaging batch means gives equal weight to full and partial batches. If you want a sample-weighted mean, accumulate each mean loss times the batch's actual size, plus the number of processed samples. Then sum both quantities across ranks.

Suggested metric addition for this unweighted criterion

```
# Reset at the start of each epoch, on every rank.
loss_sum = 0.0
sample_count = 0

# Inside the batch loop, after loss is computed.
loss_sum += loss.item() * labels.size(0)
sample_count += labels.size(0)

# After the loop, on every rank, before rank-0 logging.
stats = torch.tensor(
    [loss_sum, sample_count], dtype=torch.float32, device=device
)
if distributed:
    dist.all_reduce(stats, op=dist.ReduceOp.SUM)
global_mean_loss = (stats[0] / stats[1]).item()
if rank == 0:
    print(f"Mean training loss: {global_mean_loss:.4f}")
```

Float32 is compatible with this example's CPU, CUDA, and MPS paths; for much larger counts, keep an integer counter separately. This measures loss over processed training samples while the model changes during the epoch. It includes any examples repeated by sampler padding. It is not a loss measured once on the unique dataset at the final model state. Class weighting or ignored targets would require revisiting the denominator.

Collective rule. The all_reduce belongs outside the rank-0-only branch. Every participating rank must call it in the same logical order. A logging improvement can otherwise introduce a hang by making one rank wait for a collective that the others never enter.

12 Evaluate once and let every rank finish

The current code places a barrier after training. Rank 0 then evaluates the entire test loader and saves the figure, while the other ranks proceed to the final barrier and wait. The unsharded test loader is intentional here: only rank 0 consumes it, so accuracy covers the full test set once.

Current code inside the rank-0 evaluation block

```
eval_model = model.module if distributed else model
eval_model.eval()
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)
        logits = eval_model(images)
        predictions = torch.argmax(logits, dim=1)
```

model.module is the underlying CNN inside the DDP wrapper. Using it avoids invoking DDP's wrapper-level forward communication while the other ranks are not evaluating. This is appropriate for the current plain CNN. A future model containing its own distributed collectives needs a new evaluation plan. eval() sets layer behavior; no_grad() prevents gradient recording. They serve different purposes.

The control flow to preserve

```
Rank 0              Other ranks
finish training     finish training
       barrier 1 reached by ALL
full test pass      wait at barrier 2
plot mistakes       wait at barrier 2
       barrier 2 reached by ALL
cleanup             cleanup
```

The final dist.destroy_process_group() releases distributed resources after the second barrier. A barrier is a meeting point, not error recovery. A barrier inside if rank == 0 would wait for absent peers. A crash during plotting can leave other ranks waiting until failure or timeout. A very long evaluation can also exceed a communication timeout.

Current reporting and plotting limits

Epoch logs, evaluation, and plotting are rank-0-only. Startup diagnostics, sample counts, and shape prints are still executed by every rank. Keep rank labels for debugging or gate routine messages deliberately.

The script gathers all incorrectly classified test images on CPU, concatenates them, and indexes the first nine. Fewer than nine errors causes an indexing failure. A suggested fix is to hide all axes first and draw only min(9, len(all_incorrect_images)) examples, with a clear message if there are none. Use a noninteractive plotting backend for batch execution and save into a unique run directory.

13 Launch one node then understand two

An ordinary python invocation starts one process. torchrun starts workers, assigns their environment variables, and gives them rendezvous information. It does not allocate scheduler resources; it must run where the required GPUs are already available. [3]

Single-node example with two visible CUDA GPUs

```
torchrun --standalone --nnodes=1 --nproc-per-node=2 \
    MNIST_demo.py --learning-rate 0.001 \
    --epochs 2 --batch-size 64
```

Apply the startup and data-preparation follow-ups before using this as a multi-rank smoke test. The expected identities are rank/local_rank 0/0 and 1/1, both with world_size 2. The full global batch is 128. Both workers run the whole script, but each training loader sees its own sampler indices. Only rank 0 runs the final evaluation.

What changes across nodes

Each node needs its own launcher invocation. The launchers must agree on the number of nodes, workers per node, and rendezvous address and port. For a static homogeneous launch, node rank distinguishes the nodes; global worker rank spans them all. MASTER_ADDR must be reachable from every node, not localhost on separate machines. [3]

Conceptual static two-node template only

```
# Start once on EACH allocated node through the site launcher.
# Set these placeholders consistently before executing.
torchrun --nnodes=2 --nproc-per-node=2 \
    --node-rank=<0-on-first-node-or-1-on-second-node> \
    --master-addr=<reachable-coordinator-hostname> \
    --master-port=<agreed-available-port> \
    MNIST_demo.py --epochs 2 --batch-size 64
```

Angle-bracket values are explanatory placeholders, not shell-ready arguments. This template illustrates torchrun's static rendezvous pattern. It is not a PBS submission script and does not start remote launchers on its own.

The two nodes would contribute four workers altogether. Each process must access matching code, compatible software, and the needed data. An address/port identifies the coordination endpoint; it does not replace the GPU communication network. Port policy, network routing, and the approved way to start work on allocated nodes must be verified locally.

Self check. Does --nnodes=2 on the first node start Python on the second? No. The second node must also start its launcher through the cluster's supported mechanism.

14 Let PBS supply the allocation

PBS will surround the Python workflow with resource allocation and job management. A submission describes the requested resources, queue, walltime, environment setup, and launch action. The exact directives and multi-node launch mechanism depend on the site's PBS installation and policies.

```
PBS grants nodes and GPUs
        |
Job environment reveals allocated hosts and resources
        |
Site-approved launcher starts torchrun on each node
        |
torchrun creates workers and sets rank environment
        |
MNIST_demo.py binds devices and trains with DDP
```

Do not guess an mpiexec, pbsdsh, SSH, queue, module, or GPU-resource syntax. The Python code should mostly remain scheduler-independent; the site-specific job script translates the allocation into a correct launch. Nodefile repetition may represent CPU slots and must not be interpreted as a GPU count.

Inspect inside a real allocated job

The following are diagnostic shell commands, not a complete PBS submission. Run them inside an allocated job using your site's established submission instructions.

```
hostname
printf 'PBS_NODEFILE=%s\n' "${PBS_NODEFILE:-}"
if [ -n "${PBS_NODEFILE:-}" ] && [ -r "$PBS_NODEFILE" ]; then
    cat "$PBS_NODEFILE"
fi
env | grep '^PBS_'
if [ -n "${PBS_JOBID:-}" ]; then
    qstat -f "$PBS_JOBID"
fi
```

Capture the hostname, raw nodefile, PBS environment, and full job resource record. From the login node, qstat -f followed by the actual job ID may also expose useful allocation details. Availability, fields, and permissions vary. Inspect visible GPUs on each node as well, using the site's supported GPU diagnostic tools.

Questions the evidence must resolve

Confirm the unique allocated hosts, GPUs visible to each worker, how node ranks will be assigned, and which hostname and port can serve as the coordinator. Also determine whether ./data is shared, where the job starts, how the software environment is activated, and how logs and outputs are retained.

A nodefile containing nodeA twice and nodeB twice establishes repeated host entries, not “two GPUs per node.” A unique-host list can help identify nodes, but the allocation record and site documentation must explain the repetition and GPU resources. Tailor the final PBS launch only after these facts are known.

15 Coordinate data and harden the demo

Both datasets currently use download=True on every process. Multiple workers may try to download or extract into the same ./data directory. A robust run prepares data once before workers consume it.

Suggested pattern only when all ranks share the same data path

```
if not distributed or rank == 0:
    datasets.MNIST(root="./data", train=True, download=True)
    datasets.MNIST(root="./data", train=False, download=True)
if distributed:
    dist.barrier()

mnist_trainset = datasets.MNIST(
    root="./data", train=True, download=False, transform=transform
)
mnist_testset = datasets.MNIST(
    root="./data", train=False, download=False, transform=transform
)
```

Place this in the dataset section after distributed initialization. All ranks must reach the barrier. A failed download can still fail the job; a barrier does not handle that failure for you. Preparing datasets before launching training is often simpler, especially if compute nodes have no internet access.

If each node has private storage, global rank 0 cannot prepare every node's files. Stage the dataset on every node, or arrange one preparation process per node with appropriate coordination. The correct pattern follows the storage layout, not the GPU count.

Follow ups grounded in the current files

- Add explicit CUDA binding before NCCL setup and validate the distributed environment. The helper currently assumes LOCAL_RANK exists.
- Coordinate MNIST preparation, and make data and output locations explicit so batch working directories do not change their meaning accidentally.
- Label the current epoch loss as local, or add the all-rank, sample-weighted reduction in Section 11.
- Guard the nine-image plot and avoid incorrect.png collisions. Retain only a bounded set of mistakes for larger datasets.
- Add seeds and configuration records for reproducible comparisons. No explicit seed or checkpoint saving is present in these files.
- Move top-level execution into main() with an if __name__ == "__main__" guard before adding spawned DataLoader workers or importing the script as a module.

Order of work. First verify one device. Then fix device ownership, data preparation, and output safety. Test two workers on one node before adding a second node and PBS-specific launching. This separates model errors from coordination and scheduler errors.

16 Checklist for moving to a real model

Use this as a readiness review rather than assuming that a successful MNIST run proves every future architecture will work.

Model and learning behavior

- Verify input channels, sizes, target meaning, and output shape with a small batch. Revisit the loss contract for tasks other than ten-class classification.
- Materialize any lazy parameters before DDP. For new normalization layers, dropout, conditional branches, or unused parameters, review their distributed behavior.
- Confirm the full model and optimizer state fit on each GPU. DDP replicates them; it does not make an oversized model fit by splitting it across GPUs.
- Establish a single-device accuracy and throughput baseline. Use a validation split for hyperparameter selection; preserve the test set for final evaluation rather than repeatedly tuning against it.

Data and experiment identity

- Define transforms, splits, seeding, sharding, sampler padding, and equal training-step counts across ranks.
- Record per-rank batch size, world size, global batch size, learning rate, epochs, seed, and any gradient accumulation. Accumulation would add another factor to effective batch size; it is not in this demo.
- Isolate run outputs, save the configuration and software versions, and implement checkpoint/resume for model, optimizer, epoch, and relevant random state.

Coordination and operational checks

- Bind one process to the intended visible GPU; verify rank/local_rank/world_size in a short smoke test.
- Ensure matching collectives on all ranks. Audit rank-0 branches so logging, evaluation, or early exits cannot strand peers.
- Choose local or globally reduced metrics intentionally. If evaluation is later sharded, reduce counts and handle padding without double-counting examples.
- Check shared versus node-local storage, staged data, network access, scheduler launch conventions, and actual PBS resource fields.
- Measure scaling before optimizing. Tune loading, transfers, mixed precision, and communication only after correctness is established.

Questions you should now be able to answer

Why does each GPU need a different sampler but the same architecture? Each replica learns from different examples, while corresponding parameters must have matching meaning for gradient averaging.

Why does rank 0 logging not give a global loss? Printing chooses a speaker; reduction combines measurements. The current code only chooses a speaker.

What remains when the Python path is ready? Validate the real allocation, GPU visibility, storage, and remote launch mechanism, then construct and test the PBS wrapper for that cluster.

17 Source map and further reading

Project files

The architecture and tensor trace come from CNN2D.__init__ and CNN2D.forward in architectures.py. The environment behavior comes from get_distributed_info in utils.py. The parser, device branches, dummy forward, DDP wrapper, datasets, loaders, Adam loop, local loss reporting, rank-0 evaluation, mistake plot, barriers, and cleanup come from MNIST_demo.py.

The Cartesian-product sweep, run identifiers, metric reduction, explicit CUDA binding, coordinated download, and hardening steps are suggested extensions. The walkthrough keeps the current training structure visible instead of replacing it with a different framework.

Official PyTorch references

These references support the distributed and API explanations. Use documentation matching the PyTorch version installed on the eventual cluster. Links were checked on 14 October 2026; they do not identify the version used to run the uploaded project.

[1] Data loading and DistributedSampler. Batching, worker processes, sampler padding, and set_epoch.

https://docs.pytorch.org/docs/stable/data.html

[2] CrossEntropyLoss. Logit inputs, class-index targets, and reduction semantics.

https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

[3] torchrun. Worker creation, environment variables, single-node launching, and multi-node rendezvous.

https://docs.pytorch.org/docs/stable/elastic/run.html

[4] Distributed communication package. Process groups, environment initialization, CUDA devices, and collectives.

https://docs.pytorch.org/docs/stable/distributed.html

[5] DistributedDataParallel API. Model replication, input-sharding responsibilities, synchronization, and wrapper behavior.

https://docs.pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html

[6] Distributed Data Parallel design notes. Autograd hooks, gradient buckets, and communication overlap.

https://docs.pytorch.org/docs/stable/notes/ddp.html

What to keep beside the guide

Keep the original three Python files, the exact invocation and environment used for each experiment, a record of required follow-ups, and the allocation diagnostics from the first cluster test. Those materials connect the explanation to a reproducible run and provide the evidence needed to write the eventual PBS launcher.